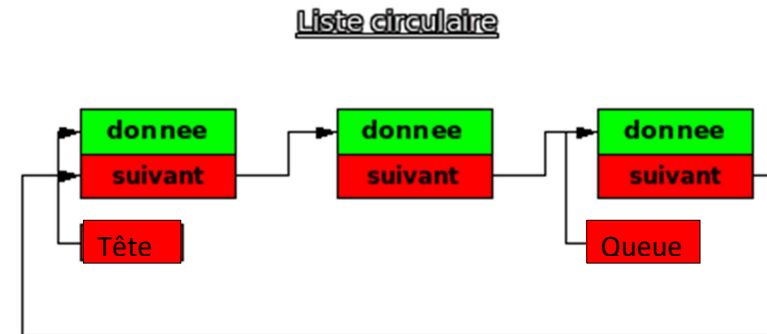


Table des matières

Liste circulaire : Amélioration de la structure	2
Structure ListeCTQ	3
Opérations liées à la structure Liste	4
Opérations liées à la structure ListeCTQ	4
Algorithmes sur les listes circulaires modifiées	5
Initialisation d'une liste vide	5
Tester si une liste est vide	6
Insertion d'un élément dans la liste	7
Insertion dans une liste vide	8
Insertion en queue dans une liste circulaire non vide	11
Suppression d'un élément dans une liste	13
Suppression en tête	15
Suppression en queue	16
Suppression dans une liste avec un seul élément	17
Suppression d'une liste circulaire	19
Parcours d'une liste circulaire	20
Solution 1 : on utilise la taille de la liste pour la condition d'arrêt.	20
Solution 2 : parcours en utilisant les deux adresses tête et queue.	21

Liste circulaire : Amélioration de la structure

Pour avoir le contrôle de la liste il est préférable de sauvegarder certains éléments : le premier élément (tête), le dernier élément (queue), (éventuellement le nombre d'éléments).



Structure ListeCTQ

```
typedef struct cellule{
    TElement donnee ;
    struct cellule *suivant ;
}*Liste ;
```

```
typedef struct{
    Liste tete ;
    Liste queue ;
    [int taille ;]
}ListeCTQ ;
```

Le pointeur **tete** contiendra l'adresse de la première cellule de la liste.
Le pointeur **queue** contiendra l'adresse de la dernière cellule de la liste.

Le membre **taille** contient le nombre de cellules.

Quelle que soit la position dans la liste, les pointeurs **tête** et **queue** pointent toujours respectivement vers la 1er et la dernière cellule.

Le membre **taille** contiendra le nombre d'éléments de la liste quelle que soit l'opération effectuée sur la liste.

Opérations liées à la structure Liste

Fonction donne : Liste → TElement

Fonction suivant : Liste → Liste

Opérations liées à la structure ListeCTQ

Fonction tete : ListeCTQ → Liste

Fonction queue : ListeCTQ → Liste

Fonction taille : ListeCTQ → Entier

Algorithmes sur les listes circulaires modifiées

Initialisation d'une liste vide

Prototype de la fonction :

Fonction : `initLCTQ : [] → ListeCTQ`

```
ListeCTQ initLCTQ(void){  
ListeCTQ l ;  
    l.tete = NULL;  
    l.queue = NULL;  
    l.taille = 0;  
    return l ;  
}
```

Tester si une liste est vide

Fonction `estVideLCTQ : ListeCTQ → Booléenne`

Fonction `estVideLCTQ(l) → Booléenne`

Debut

Renvoyer `taille(l)=0`

Fin

Insertion d'un élément dans la liste

Fonction **insertion** : TElementxListeCTQ → ListeCTQ

Données

elt : TElement

l : ListeCTQ

Traitement : $l \leftarrow l + e$

Résultat

L : ListeCTQ

Principe est :

- Allocation de la mémoire pour le nouvel élément
- Remplissage du membre donné de la nouvelle cellule
- Déterminer l'adresse d'insertion
- Mise à jour des chaînages.
- Cas particulier : dans une liste avec un seul élément, le 1er est en même temps le dernier.
- Mettre à jour la taille de la liste

Insertion dans une liste vide

Prototype de la fonction :

ListeCTQ inserLCTQV(TElement elt, ListeCTQ l) ;

Etapes :

- Allocation de la mémoire pour le nouvel élément.
- Remplissage du membre donné de la nouvelle cellule.
- Réalisation des chaînages :
 - Le pointeur **suivant** du nouvel élément pointera vers lui-même (*c'est l'étape qui rend la liste circulaire*)
 - Les pointeurs **tête** et **queue** pointeront vers le nouvel élément
- la taille est mise à jour

Exemple

```
// Allocation mémoire et remplissage de la
cellule avec l'élément à insérer.
```

Fonction allocMemRempCel : TElement → Liste

```
Liste allocMemRempCel(TElement elt){
    Liste cel ;

    cel=(Liste)malloc(sizeof(struct cellule)) ;

    // tester l'allocation

    If (estVide(cel))

        rerror(« pb d'alloc mémoire) ;

    cel->donnee=elt ;

    return cel ;

}
```

Insertion dans une liste vide

// Fonction d'insertion

```
ListeCTQ inserListeCTQV(TElement elt, ListeCTQ l){
    Liste cel;

    cel = allocMemRempCel(elt) ;
    cel->suivant = cel;
    l.tete = cel;
    l.queue = cel;
    l.taille++;
    return l ;
}
```

/* La procédure d'insertion dans une liste vide */

```
void inserListeCTQV(TElement elt, ListeCTQ *l) {
    Liste cel;

    cel = allocMemRempCel(elt) ;
    cel->suivant = cel;
    l->tete = cel;
    l->queue = cel;
    l->taille++;
}
```

Insertion en queue dans une liste circulaire non vide

Prototype de l'algorithme :

```
void inserQueueLCTQ(TElement elt, ListeCTQ *l) ;
```

```
ListeCTQ inserQueueLCTQ(TElement elt, ListeCTQ *l) ;
```

Etapes :

- Allocation mémoire d'une nouvelle cellule pour le nouvel élément
- Remplissage du membre donné par l'élément de la nouvelle cellule.
- *Réalisation* des chaînages (A), (B) et (C) :
 - le pointeur **suivant** de la nouvelle cellule pointe vers la tête de la liste (*garder la liste circulaire*)
 - le pointeur **tête** ne change pas
 - le suivant de la queue pointe vers la nouvelle cellule
 - le pointeur **queue** pointe vers la nouvelle cellule
- la taille est incrémentée d'une unité

Exemple

```
/* L'algorithme d'insertion en queue dans une
liste non vide */
```

```
void inserQueueLCTQ(TElement elt, ListeCTQ *l){
    Liste cel, ll;
    //allocation mémoire & remplissage
    cel = allocMemRempCel(elt) ;
    cel->suiv = suivant(queue(*l)); // chaînage A
                                // ou tete(*l)
    (l->queue)->suivant = cel;      // chaînage B
    l->queue = cel;                 // chaînage C
    l->taille++;
}
```

Insertion en tête dans une liste circulaire non vide

Prototype de l'algorithme :

```
void inserTeteLCTQ(TElement elt, ListeCTQ *l) ;
```

```
ListeCTQ inserTeteLCTQ(TElement elt, ListeCTQ *l) ;
```

Suppression d'un élément dans une liste

Pour supprimer un élément dans la liste, il y a plusieurs situations :

- 1) Suppression en tête
- 2) Suppression en queue
- 3) Suppression par position
- 4) Suppression par valeur
- 5) Suppression dans la liste
- 6) ...

Principe de suppression d'un élément de la liste :

- Déterminer l'adresse de suppression
- Préserver la cellule contenant l'élément à supprimer
- Modification des chaînages
- Libérer la mémoire occupée par la cellule à supprimer
- Mettre à jour la taille de la liste

Suppression en tête

Hypothèse : Taille(lc) >= 2

Prototype de la fonction :

```
void suppTeteListeCirc(ListeCTQ *l){
```

Etapes:

- Préservation de l'adresse de la tête
- Modification des chaînages :
 - le pointeur **tête** pointera vers suivant (tete)
 - le pointeur **suivant** de la queue la tête pointera vers la nouvelle tête.
- La taille de la liste sera décrémentée d'un élément
- Libération mémoire occupée par la tête

Exemple

Suppression en tête

Hypothèse : Taille(lc) >= 2

```
void suppTeteLCTQ(ListeCTQ *l){  
    Liste cel;  
    cel = tete(*l);  
    l->tete = suivant(tete(*l));  
    l->queue->suivant = tete(*l);  
    free(cel);  
    l->taille--;  
}
```


Suppression dans une liste avec un seul élément

Prototype de la fonction:

```
Void supPEltUniqueLCTQ(ListeCTQ *l);
```

Étapes:

- Préservation de l'adresse de la tête de liste
- Le pointeur **tête** pointera vers **NULL**
- Le pointeur **queue** pointera vers **NULL**
- Libération mémoire
- La taille de la liste est réduite à 0

Exemple

La fonction

```
/* suppression dans une liste avec un seul élément*/  
Void supPEltUniqueListCirc(ListeCTQ *l){  
    Liste cel;  
    cel = tete(l);  
    l->tete = NULL;  
    l->queue = NULL;  
    free(cel);  
    l->taille--; // l->taille = 0  
}
```

Suppression d'une liste circulaire

Fonction supListCirc : ListeCircModif → {}

Principe : Pour détruire la liste circulaire entièrement, on utilise l'algorithme de suppression en tête dans une liste circulaire tant que la taille est plus grande que 1, ensuite on applique l'algorithme de suppression dans une liste circulaire avec un seul élément.

/* La fonction détruit la liste */

```
Void supListCTQ(ListeCTQ *l){
    while ( taille(*l) > 1){
        supTeteLCTQ(l);
    }
    supEltUniqueListCirc(l) ;
}
```

Parcours d'une liste circulaire

Parcours de la liste de la première cellule (définie par le membre tête), jusqu'à attendre la dernière cellule (définie par le membre queue)

Parcours tête à queue (du 1^{er} vers la dernière cellule)

Solution 1 : on utilise la taille de la liste pour la condition d'arrêt.

```
void itParcours(ListeCTQ l){
    Liste ll;
    int lg
        ll = tete(l); //Cellule courante
        lg = taille(l);
        for(i=0; i<lg; i++){
            traiter(donnee(ll));
            ll = suivant(ll);
        }
}
```

Solution 2 : parcours en utilisant les deux adresses tête et queue.

Hypothèse : on suppose que la liste est non vide

```
/* Parcours entre la tête et la queue*/  
void itParcours2(ListeCTQ l){  
    Liste cc, ca;  
    cc = tete(l);    //Cellule courante  
    ca = queue(l) ;  // Cellule d'arrêt  
    do{  
        traiter(donnee(cc));  
        cc = suivant(cc);  
    } while (cc !=ca) ;  
    if(tete(l)!=ca) traiter(donnee(ca));  
}
```